
RV32IM Single Cycle Processor

Samiksha Aveg
Department of *Electrical and Computer Engineering*
The University of Sydney
Sydney, Australia
510574673

Abstract— This paper presents the development of open-source RISC-V (RV32I) Single Cycle Processor ISA modified to include the “M”-Extension Instruction set. The Hardwired Control Unit makes the micro-architecture simple and efficient for the predictable instructions. The test programs created to assess the processor are also presented in this paper. The proposed processor is implemented in Verilog HDL and further is optimised to fit on the FPGA board. This is found that the maximum clock frequency is 32 MHz and total cycle counts for each test program are reported in the simulation output.

Keywords— RV32I, “M”-Extension, FPGA, processor, test program

I. INTRODUCTION

RISC-V is a new processor architecture consisting of instruction sets designed to support computer architecture research and education. It contains integer computational instructions, integer loads, integer stores, and control-flow instructions. It mainly has two base integer variants: 32-bit and 64-bit address space variations supporting hardware and software development. For this assignment, the RISC-V 32-bit variant is used as it is believed to remain adequate for many of the current embedded systems and client devices and is desirable for its lower memory traffic and energy consumption. In addition, 32-bit address spaces are sufficient for educational purposes as they can be designed to include more

optional instruction-set extensions. An extension may be categorised as either standard, custom or non-confirming and a set of standard extensions are defined to provide integer multiplication/division, atomic operations and single or double-precision floating point arithmetic. The aim of this assignment is to implement one of these standard extensions, namely “M” Extension which contains an instruction-set to multiply and divide values held in the integer registers.

II. BACKGROUND

A basic NERV program is provided implementing base integer operations including addition, subtraction, branch, jump and more. This single-cycle RV32I processor is modified by implementing “M” extensions to its simple pipeline. The standard integer instructions are enhanced to include multiplication (MUL, MULH, MULHU, MULHSU) and division (DIV, DIVU, REM, REMU) operations, allowing for the processing of both signed and unsigned values stored in the integer registers. Significant research and efforts have been dedicated to modifying and optimising the program so that it easily fits the FPGA. This investigation revealed that Booth's Algorithm would be highly beneficial for executing the “M” Extension instructions by performing bitwise shifts combined with additions and subtractions.

A comprehensive baseline performance analysis is performed on a single-cycle 32-bit processor, as outlined in the subsequent sections.

III. ARCHITECTURE

In general, the pipeline can be divided into five stages: Instruction Fetch, Instruction Decode, Instruction Execute, Memory Access and Write-Back. Each of these stages is essential for implementing control logic that guarantees accurate reading and processing of inputs. This structured approach enables the execution of specific operations and the generation of corresponding outputs with precision. The Instruction Fetch (IF) block plays a crucial role in retrieving instructions, while the Instruction Decode (ID) block focuses on decoding these fetched instructions and generating the necessary control logic. In the Instruction Execute (IE) block, this control logic is directed to the ALU, which carries out the required operations. If the instruction necessitates memory access for execution, it proceeds to the Memory Access (MA) block to retrieve the needed data. Finally, the Write-Back block is responsible for storing the results of the operations performed in the earlier stages back into the register file. This architecture follows the conventional encodings given below (Figure 1) to execute instructions:

31	27	25	24	20	19	15	14	12	11	7	6	0	
FUNCT7		RS2		RS1		FUNCT3		RD		OPCODE			R-TYPE
IMM[11:0]				RS1		FUNCT3		RD		OPCODE			I-TYPE
IMM[11:5]		RS2		RS1		FUNCT3		RD		OPCODE			S-TYPE
IMM[12/10:5]		RS2		RS1		FUNCT3		RD		OPCODE			SB-TYPE
IMM[31:12]								RD		OPCODE			U-TYPE
IMM[20/10:11/11/19:12]								RD		OPCODE			UJ-TYPE

Fig. 1: Instruction Encodings of the types

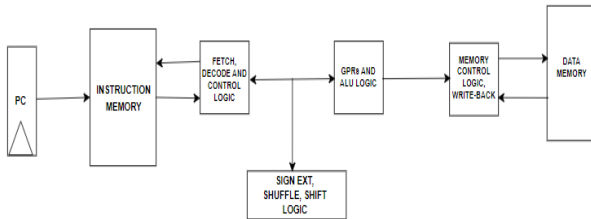


Fig. 2: Overview of RV32IM Datapath

Figure 2 broadly represents the datapath of the RV32IM processor. The detailed version of this datapath can be seen in the Appendix (Figure 3) along with the Hardwired Control Table (Figure 4).

The Hardwired Control Unit is an essential part of this design, allowing for efficient execution for predictable instructions.

The *fetch, decode and control logic* has the responsibility to fetch the 32-bit address spaced instructions from the instruction memory, decode these instructions as per the standard encodings displayed before and generate different control signals for proper execution of instructions. It also plays a crucial role in carrying out branch and jump operations effectively.

The *program counter* is essential for managing the control flow of the processor and it features a dedicated adder which increments the *program counter* by 4 with each clock cycle, ensuring efficient operation.

The *GPRs and ALU Logic* block consists of 31 general purpose registers, and the ALU (Arithmetic Logic Unit) is responsible for managing the read and write logic as well as performing arithmetic computations involving pairs of registers or register-immediate values. For branch and jump instruction sets, the ALU is not directly involved in the target address calculations, it provides the calculated results to the Control Unit which then executes target address selection logic and decides whether the branch is to be taken. There are primarily 5 types of instructions: I, S, B, U and J-type immediate. The instruction sets under these categories are decoded according to the encodings illustrated in Figure 1. This involves executing arithmetic and bitwise shifts with the help of dedicated sign-extension and reordering modules to ensure accurate handling of both signed and unsigned operands. This decode process is accomplished through a combination of *Sign Extension, Shuffle, and Shift Logic*.

Once decoded, the instructions are reordered and distributed to various modules within the processor. For instance, for register-immediate operations the sign-extended operand is supplied to the ALU, while the second operand is retrieved from the Register File. The instructions involving memory-access are governed by *Memory Control Logic and Write-Back* module where it manages retrieval of data from data memory and storing it in register files or vice versa (store/load operations). After respective tasks are executed, the updated values are reflected in memory.

IV. RESULTS

Given that this processor operates on a single-cycle basis, it executes each instruction within a single clock cycle, resulting in a straightforward and efficient design. After the modifications are implemented on the original design, it is tested with a range of test programs tailored to evaluate different sets of inputs. The Appendix Section (Figure 5) features a detailed table of test programs designed for your convenience. This table outlines the test programs, their functionalities, expected outcomes, and rationale behind their correctness. These programs evaluate the accuracy of all the instruction sets present in the RV32IM ISA (I & M Extension) by employing the GTKWave simulation model. This allows for a detailed examination of waveforms and the analysis of various component values at each clock cycle. These simple tests provide good coverage to assess the precision of the operations, allowing for easy adjustments until the desired outcome is achieved.

Once the fundamental tests have been successfully passed, we can proceed to evaluate the results of the more advanced tests and make any necessary adjustments to the processor's datapath to meet our requirements. The processor's maximum clock frequency for executing various operations is around 38.00 MHz, as indicated in the log file (Figure 6). Moreover, the simulation outputs (Figure 7) provide a comprehensive breakdown of performance metrics, including the minimum clock period (period), x10 register value (x10), total cycle count (cycles), overall execution time (extime), execution time normalised to the original design (normextime), and the geometric mean of normextime required for the completion of each test program.

V. DISCUSSION

The process of implementing “M”- Extension Instruction Set Architecture (ISA) after extensive research is by implementing Booth's Algorithm. Booth's Algorithm is widely used for performing binary multiplication in hardware implementations to enhance efficiency, speed and accuracy. In Booth's Algorithm, the initial step involves assigning the two operand values—referred to as *rs1* and *rs2* according to the opcodes—to logical variables: *val1* (the *multiplicand*) and *val2* (the *multiplier*). Subsequently, a

64-bit variable named *product* is instantiated and initialised to 0, which will hold the product of these two operands. Since, we already know that the maximum length of the operands is 32 *bits*, the algorithm will require a maximum of 32 cycles to complete its operation.

In each iteration of the loop, we examine the least significant bit (LSB) of *val2*. If the LSB is *zero*, we simply right shift the bits of the multiplier by one position. However, if the LSB is one, we left shift the bits of the multiplicand based on the current loop index and add the result to the product. The loop continues until the multiplier reaches zero, at which point we update the destination register (*rd*) with the final product value. Similarly, division operations can be performed by substituting addition with subtraction and instead of least significant bit, most significant bit is used. The flowchart below (Figure 8) presents the process in a significantly clearer and more comprehensible way.

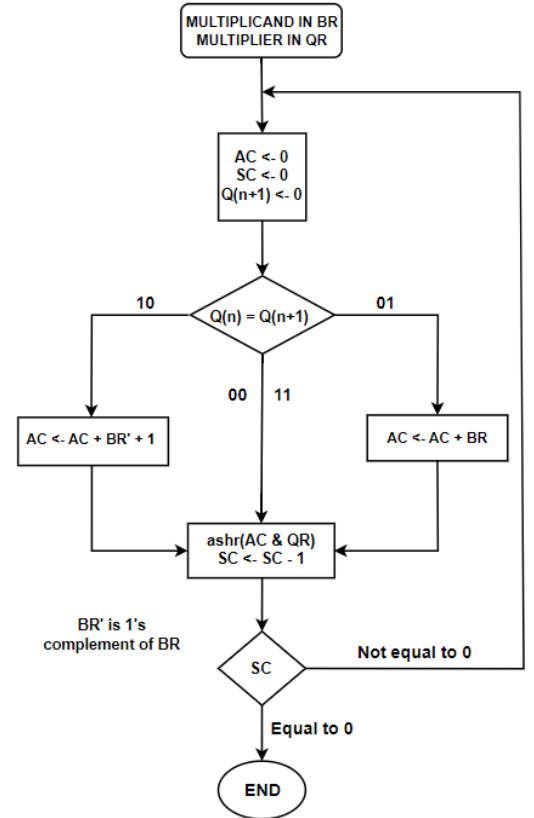


Fig. 8: Flowchart of the Booth's Algorithm

Flag variables are established to trigger specific instructions when set to one. Once all arithmetic and logical operations are executed, these flag variables are reset to zero.

VI. CONCLUSION

The single-cycle RISC-V 32-bit variant has been successfully enhanced to include the 'M' Extension instruction set with the help of extensive research and efforts. This was validated with the help of tests created and then comparing the actual outcome against our anticipated results. Consequently, for a diverse range of input values, the outcomes were consistently accurate, demonstrating the integrity of the implementation.

The author presented their effort towards creating an optimised RV32IM architecture designed for seamless integration onto an FPGA. The current processor hopes to facilitate future implementations of both specific and general applications within the realm of IoT and real-world embedded systems. Future goals encompass improving the microarchitecture style and integrating with a floating point co-processor with the aim of implementing remaining extensions to significantly improve efficiency. These results may be featured in an upcoming paper aimed at education, research, and industry, while simultaneously paving the way for creating opportunities for faster designs.

VII. REFERENCES

- [1] unpriv-isa-asciidoc_20240411.pdf, “unpriv-isa-asciidoc_20240411.pdf,” *Google Docs*, 2019.
<https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj7Omv8tFtkp/view?pli=1> (accessed Oct. 13, 2024).
- [2] B. Giovino, “How the RISC-V Multiply Extension Adds an Efficient 32-bit Multiply to the RV32I,” *DigiKey*, Nov. 08, 2021.
<https://www.digikey.com/en/blog/how-the-risc-v-multiply-extension-adds-an-efficient-32-bit> (accessed Oct. 13, 2024).
- [3] Z. Rafique, “RISC-V M-Extension Support for SERV - Zeeshan Rafique - Medium,” *Medium*, Aug. 21, 2021.
<https://medium.com/@zrafique/gsoc21-with-fossi-foundation-ff1c869daec8> (accessed Oct. 13, 2024).
- [4] Electronics etc, “Multipliers,” *Electronics etc...*, Aug. 12, 2018.
<https://tomverbeure.github.io/rtl/2018/08/12/Multipliers.html#risc-v-multiply-instructions> (accessed Oct. 13, 2024).
- [5] “Booth’s Multiplication Algorithm,” *GeeksforGeeks*, Apr. 03, 2018.
<https://www.geeksforgeeks.org/booths-multiplication-algorithm/>
- [6] J. Doe, “How do I implement multiplication and division in MIPS assembly without using the built in instructions?,” *Stack Overflow*, 2024.
<https://stackoverflow.com/questions/9798813/how-do-i-implement-multiplication-and-division-in-mips-assembly-without-using-th> (accessed Oct. 13, 2024).
- [8] Board Infinity, “Booth’s Algorithm | Board Infinity,” *Board Infinity*, Feb. 2023.
<https://www.boardinfinity.com/blog/booths-algorithm/> (accessed Oct. 13, 2024).
- [9] Behrooz Parhami, *Computer Arithmetic*. 2010.

Figure 4: A Hardwired Control Table

INSN	Op2 Sel	ALU FUNC	WB Sel	Reg WriteEn	MemWrite	PC Sel
ADD	RS2	ADD	ALU	T	F	PC+4
SUB	RS2	SUB	ALU	T	F	PC+4
SLL	RS2	SLL	ALU	T	F	PC+4
SLT	RS2	SLT	ALU	T	F	PC+4
SLTU	RS2	SLTU	ALU	T	F	PC+4
XOR	RS2	XOR	ALU	T	F	PC+4
SRL	RS2	SRL	ALU	T	F	PC+4
SLL	RS2	SLL	ALU	T	F	PC+4
SRA	RS2	SRA	ALU	T	F	PC+4
OR	RS2	OR	ALU	T	F	PC+4
AND	RS2	AND	ALU	T	F	PC+4
ADDI	IMI	ADD	ALU	T	F	PC+4
SLTI	IMI	SLT	ALU	T	F	PC+4
SLTIU	IMI	SLTU	ALU	T	F	PC+4
XORI	IMI	XOR	ALU	T	F	PC+4

ORI	IMI	OR	ALU	T	F	PC+4
ANDI	IMI	AND	ALU	T	F	PC+4
SLLI	IMI	SLL	ALU	T	F	PC+4
SRLI	IMI	SRL	ALU	T	F	PC+4
SRAI	IMI	SRA	ALU	T	F	PC+4
LW	IMI	ADD	MEM	T	F	PC+4
SW	IMS	ADD	X	F	T	PC+4
BEQ	IME	X	X	F	N	PC+4/BA
BNE	IME	X	X	F	N	PC+4/BA
BLT	IME	X	X	F	N	PC+4/BA
BGE	IME	X	X	F	N	PC+4/BA
BLTU	IME	X	X	F	N	PC+4/BA
BGEU	IME	X	X	F	N	PC+4/BA
JAL	IMJ	X	PC+4	T	N	JA
JALR	IMI	X	PC+4	T	N	JRA
MUL	RS2	MUL	ALU	T	F	PC+4
MULH	RS2	MULH	ALU	T	F	PC+4
MULHSU	RS2	MULHSU	ALU	T	F	PC+4

MUL HU	RS 2	MUL HU	AL U	T	F	PC +4
DIV	RS 2	DIV	AL U	T	F	PC +4
DIV U	RS 2	DIVU	AL U	T	F	PC +4

Figure 5: Test Program Table

Test Program	“M” Instruction tested	Functionality	Outcome & Reasons
Test1: addi x10, zero, 0 li x2, -65 addi x3, zero, -1 mul x10, x3, x2 ebreak	mul	Tests two negative integers	Expected Outcome : x10 = 65 Reason: the lower 32-bits of the product of (-65) * (-1) is equal to 65.
Test2: addi x10, zero, 0 li x2, 20 addi x3, zero, 1024 mul x10, x3, x2 ebreak	mul	Tests two positive integers	Expected Outcome : x10 = 20480 Reason: the lower 32-bits of the product of 20 * 1024 is equal to 20480.
Test3: addi x10, zero, 0 li x2, 60 addi x3, zero, -1 mul	mul	Tests negative -positive integers	Expected Outcome : x10 = -60 Reason: the lower

x10, x3, x2 ebreak			32-bits of the product of 60 * (-1) is equal to 60.
Test4: addi x10, zero, 0 li x2, -65 addi x3, zero, -1 mulh x10, x3, x2 ebreak	mulh	Tests two negative integers	Expected Outcome : x10 = 65 Reason: the upper 32-bits of the product of (-65) * (-1) is equal to 65.
Test5: addi x10, zero, 0 li x2, 650 addi x3, zero, 0 mulh x10, x3, x2 ebreak	mulh	Tests two positive integers	Expected Outcome : x10 = 0 Reason: the upper 32-bits of the product of 650 * 0 is equal to 0.
Test6: addi x10, zero, 0 li x2, -36 addi x3, zero, 7 mulh x10, x2, x3 ebreak	mulh	Tests negative -positive integers	Expected Outcome : x10 = -1 Reason: the upper 32-bits of the signed

			product of (-36) * 7 = -252 is equal to -1.
Test7: addi x10, zero, 0 li x2, 4 addi x3, zero, -44 mulhsu x10, x3, x2 ebreak	mulhs u	Tests negative -unsigned integers	Expected Outcome : x10 = -1 Reason: the upper 32-bits of the signed product of (-44) * (-4) = -176 is equal to -1.
Test8: addi x10, zero, 0 li x2, 16 addi x3, zero, 12 mulhsu x10, x3, x2 ebreak	mulhs u	Tests two positive integers	Expected Outcome : x10 = 0 Reason: the upper 32-bits of the signed product of (12) * (16) = 192 is equal to 0.
Test9: addi x10, zero, 0 li x2, 57 addi x3, zero, 8 mulhu	mulhu	Tests two positive integers	Expected Outcome : x10 = 0 Reason: the upper 32-bits

x10, x3, x2 ebreak			of the unsigned product of $57 * 8 = 456$ is equal to 0.
Test10: addi x10, zero, 0 li x2, 57 addi x3, zero, 8 div x10, x2, x3 ebreak	div	Tests two positive integers	Expected Outcome : $x10 = 7$ Reason: $57/8$ returns quotient = 7 with remainder = 1. Thus, output is equal to quotient = 7
Test11: addi x10, zero, 0 li x2, -64 addi x3, zero, 8 div x10, x2, x3 ebreak	div	Tests negative - positive integers	Expected Outcome : $x10 = 65$ Reason: $-64/8$ returns quotient = -8 with remainder = 0. Thus, output is equal to quotient = -8
Test12: addi x10, zero, 0 li x2, 7 addi x3, zero, 0	div	Tests edge case of divide by zero	Expected Outcome : $x10 = -1$ Reason: Division

div x10, x3, x2 ebreak			by zero isn't possible and by default returns -1 as output.
Test13: addi x10, zero, 0 li x2, -63 li x3, -1 div x10, x2, x3 ebreak	div	Tests two negative integers	Expected Outcome : $x10 = 63$ Reason: $-63/-1$ returns quotient = 63 with remainder = 0. Thus, output is equal to quotient = 63
Test14: addi x10, zero, 0 li x2, 64 addi x3, zero, 8 divu x10, x3, x2 ebreak	divu	Tests two positive integers	Expected Outcome : $x10 = 0$ Reason: $8/64 = \frac{1}{8}$ but unsigned division returns output rounded towards zero. Thus, output is equal to 0
Test15: addi x10, zero,	rem	Tests two positive	Expected Outcome

0 li x2, 57 addi x3, zero, 8 rem x10, x2, x3 ebreak		integers	: x10 = 1 Reason: 57/8 returns remaind er = 1. Thus, output is equal to remaind er = 1
Test16: addi x10, zero, 0 li x2, -127 addi x3, zero, 8 rem x10, x2, x3 ebreak	rem	Tests negative -positive integers	Expecte d Outcome : x10 = -7 Reason: -127/8 returns quotient = -15 and remaind er = -7. Thus, output = -7
Test17: addi x10, zero, 0 li x2, 64 addi x3, zero, 8 remu x10, x3, x2 ebreak	remu	Tests two positive integers	Expecte d Outcome : x10 = 8/64 = 8 Reason: 8/64 returns remaind er = 8 Thus, output is equal to 8.

Figure 6: Booth's Algorithm Flowchart

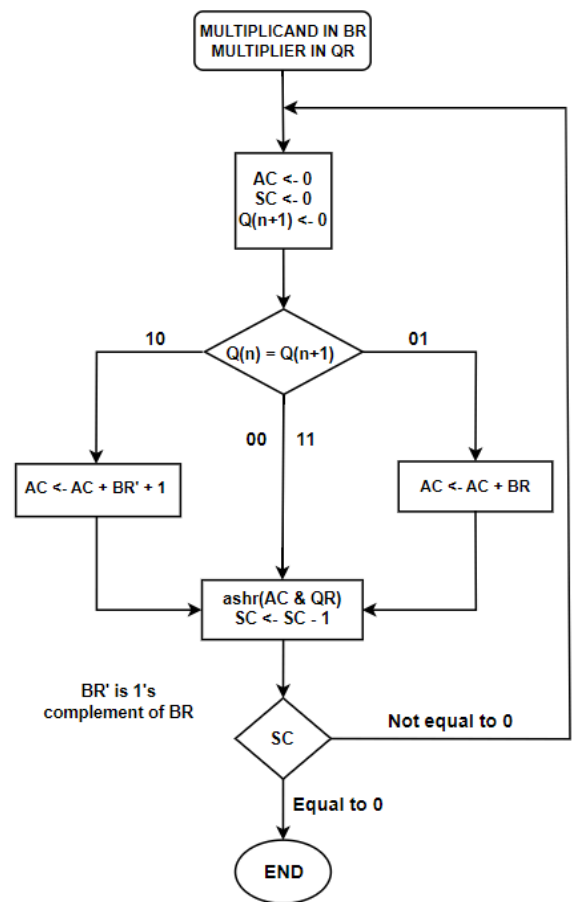


Figure 7: Log File

```
Info: Max frequency for clock 'clock$SB_IO_IN_$glb_clk': 37.29 MHz (PASS at 12.00 MHz)

Info: Max delay <async>                -> <async>                : 31.03 ns
Info: Max delay <async>                -> posedge clock$SB_IO_IN_$glb_clk: 30.19 ns
Info: Max delay posedge clock$SB_IO_IN_$glb_clk -> <async>        : 27.67 ns

Info: Slack histogram:
Info:  legend: * represents 13 endpoint(s)
Info:      + represents [1,13) endpoint(s)
Info: [ 52298, 53794) |***+
Info: [ 53794, 55290) |*****+
Info: [ 55290, 56786) |*****+
Info: [ 56786, 58282) |*****+
Info: [ 58282, 59778) |*****+
Info: [ 59778, 61274) |*****+
Info: [ 61274, 62770) |*****+
Info: [ 62770, 64266) |*****+
Info: [ 64266, 65762) |*****+
Info: [ 65762, 67258) |***+
Info: [ 67258, 68754) |***+
Info: [ 68754, 70250) |*****+
Info: [ 70250, 71746) |*****+
Info: [ 71746, 73242) |+
Info: [ 73242, 74738) |*****+
Info: [ 74738, 76234) |***+
Info: [ 76234, 77730) |*****+
Info: [ 77730, 79226) |***+
Info: [ 79226, 80722) |***+
Info: [ 80722, 82218) |*****+
201 warnings, 0 errors
```

Figure 8: Simulation Output

```
Info: Program finished normally.
python extime.py nerv-pnr.log *.result
Test summary
period: 2.681684097613301e-08
test1.result: x10=55 cycles=18 extime=4.827031375703942e-07 normextime=0.9927594529364441
test10.result: x10=1 cycles=10 extime=2.681684097613301e-07 normextime=0.2681684097613301
test11.result: x10=2147483655 cycles=10 extime=2.681684097613301e-07 normextime=0.2681684097613301
test12.result: x10=4294967290 cycles=10 extime=2.681684097613301e-07 normextime=0.2681684097613301
test13.result: x10=0 cycles=10 extime=2.681684097613301e-07 normextime=0.2681684097613301
test14.result: x10=0 cycles=10 extime=2.681684097613301e-07 normextime=0.2681684097613301
test15.result: x10=0 cycles=10 extime=2.681684097613301e-07 normextime=0.2681684097613301
test16.result: x10=0 cycles=10 extime=2.681684097613301e-07 normextime=0.2681684097613301
test17.result: x10=0 cycles=10 extime=2.681684097613301e-07 normextime=0.2681684097613301
test18.result: x10=0 cycles=10 extime=2.681684097613301e-07 normextime=0.2681684097613301
test19.result: x10=4294967295 cycles=10 extime=2.681684097613301e-07 normextime=0.2681684097613301
test2.result: x10=3790353928 cycles=15 extime=4.022526146419952e-07 normextime=0.9927594529364441
test20.result: x10=4294967295 cycles=10 extime=2.681684097613301e-07 normextime=0.2681684097613301
test21.result: x10=0 cycles=10 extime=2.681684097613301e-07 normextime=0.2681684097613301
test3.result: x10=45 cycles=40 extime=1.0726736390453204e-06 normextime=0.9927594529364441
test4.result: x10=210 cycles=130 extime=3.4861893268972914e-06 normextime=0.9927594529364441
test5.result: x10=21 cycles=942 extime=2.5261464199517296e-05 normextime=0.9927594529364441
test6.result: x10=65 cycles=10 extime=2.681684097613301e-07 normextime=0.2681684097613301
test7.result: x10=20480 cycles=10 extime=2.681684097613301e-07 normextime=0.2681684097613301
test8.result: x10=59 cycles=10 extime=2.681684097613301e-07 normextime=0.2681684097613301
test9.result: x10=4294967231 cycles=10 extime=2.681684097613301e-07 normextime=0.2681684097613301
Geometric mean=0.36622637654328144
rm test16.hex test19.elf test15.elf test21.hex test10.hex test3.hex test16.elf test19.hex test13.hex test20.hex test1
2.elf test20.elf test7.hex test9.elf test17.elf test1.hex test21.elf test18.hex test4.elf test6.elf test4.hex test14.
elf test8.hex test12.hex test5.elf test6.hex test2.hex nerv.json test8.elf test11.elf test15.hex test17.hex test9.hex
test2.elf test13.elf test5.hex test1.elf test14.hex test11.hex test18.elf test7.elf test10.elf test3.elf
```